FPGA Lab – Introduction to MicroBlaze

The aim of the first FGPA laboratory session is familiarize you the AMD tools, namely Vivado for developing hardware platforms and Vitis for writing C software, and the MicroBlaze CPU.

Objectives

By the end of the session you should be able to use to starter code from moodle to:

1. Set up a Vivado project targeting the Zynq 7000 All Programmable System On Chip (APSoC).
2. Use Vivado to instantiate a minimal MicroBlaze configuration, and generate the hardware platform.
3. Use Vitis to set up the software development environment and run a hello world C application on MicroBlaze

Background

Microblaze [1] is a customisable Intellectual property (IP) core that implements a RISC microprocessor. Figure 1 illustrates the key components, with shaded block denoting optional parts.

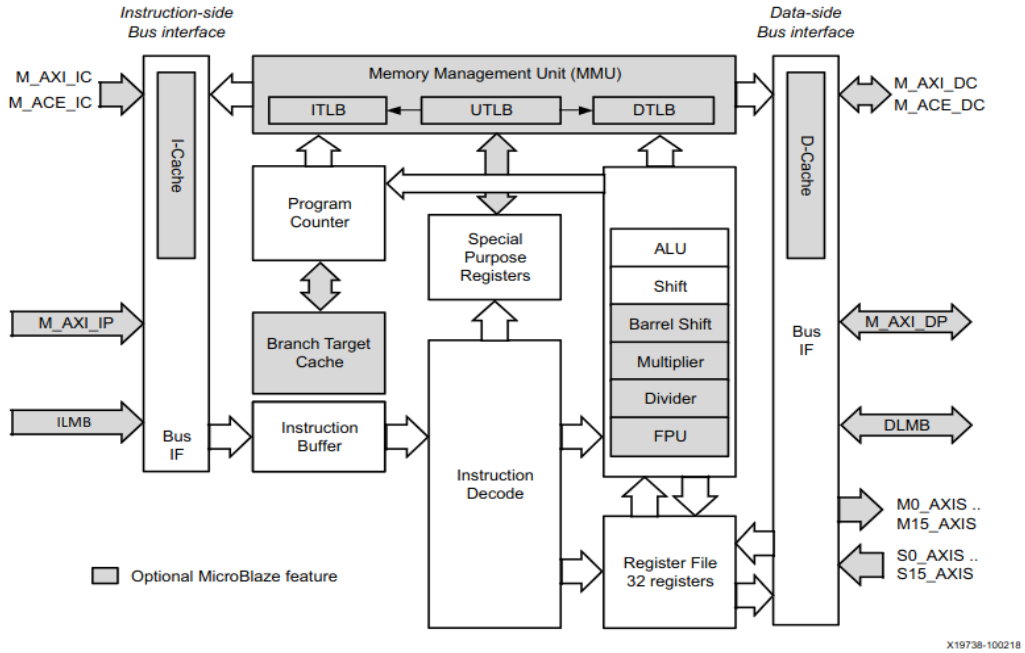

Figure 1: MicroBlaze block diagram

MicroBlaze is designed for embedded applications, thus some architectural features are best understood through this perspective. Here is a succinct list of the key points:

- Single issue in-order pipelined processors with configurable number of stages (3/5/8)
- Optional integer and floating point unit
- Local memory bus with single cycle latency and optional AXI4 instruction and data caches
- Optional AXI4 (lite) memory-mapped IO Bus
- Optional memory management unit, capable of hosting operating system

The MicroBlaze Processor Reference Guide (UG984) [1] contains detailed description of the processor's organization, micro architecture, as well as, information relating to its programming.

Using Microblaze with AMD Tools

AMD provides many tools to develop FPGA-based projects. For our labs we will be using **Vivado** to build hardware platforms, and **Vitis** to write software for them.

First we will design and build the hardware platform. Using a block-based approach Vivado allows us to define a small System-on-Chip, and configure key architectural features at high level of abstraction (i.e. using the GUI).

For the second part, we will move to a software development environment where using Vitis we'll write C software to run applications on the hardware platform. Vitis is also used to program the FPGA board and debug the running program.

The AMD documentation is comprehensive. There can you read further about the block-based approach in Vivado[2 p.6], and the software development flow in Vitis [3 p.18].

Implementation guidelines during the Lab

Download the source files for Lab 0 from moodle, then follow the next steps to set up a minimal Vivado project.

1. Launch Vivado
2. Using the TCL console in the welcome screen change directory (cd) to lab0.
3. Source build_bd.tcl located in the scripts directory with:
   `source ./scripts/build_bd.tcl`
4. The expected content in the block diagram is shown in Figure 2.

The script will set up a project for the target part (XC7Z020-1CLG400C). Afterwards, it creates a block diagram and instantiates key IP blocks for clocking and reset. IO ports are created (consistent with the constraints in the `.xdc` file) to bring the clock and reset signals from the board into the FPGA.

A counter is additionally created, and tied to a board LED to visually communicate the presence of the clock signal. The reference manual [4] and board schematics [5] contain relevant information for these

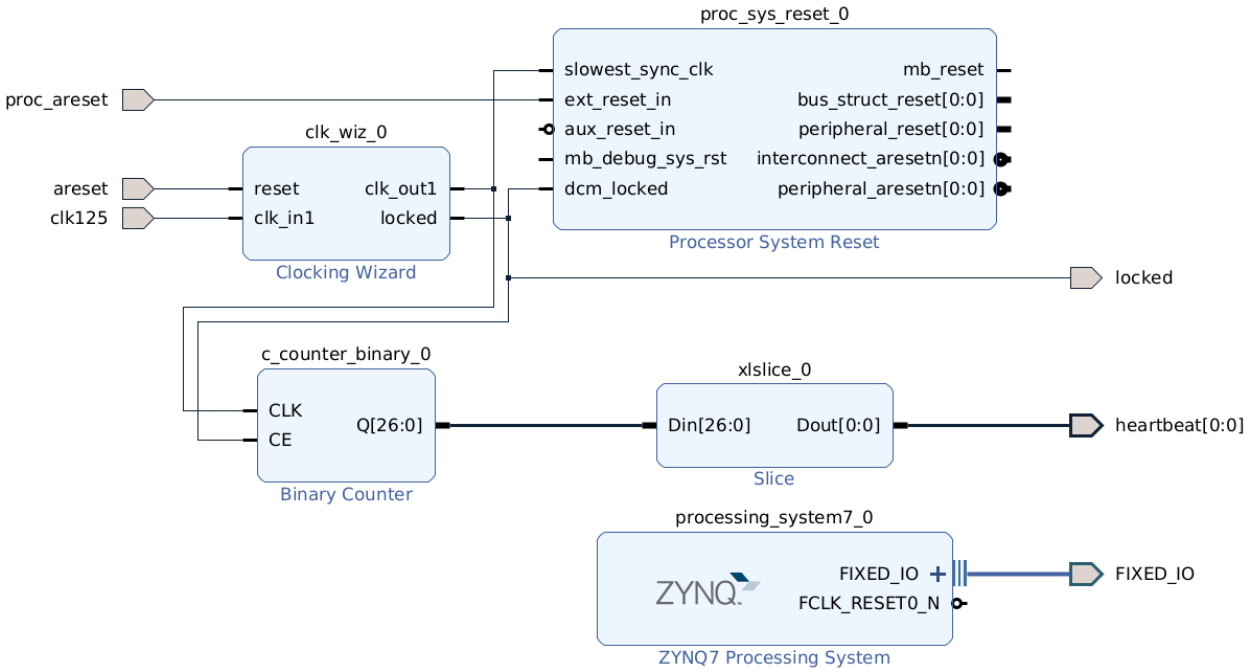

Figure 2: Expected block design view after running build_bd.tcl

Note: While we will not be using the zynq block, it must be present in the design for the programming tools to function correctly.

Adding MicroBlaze

Here we instantiate the MicroBlaze processor and using the Vivado's block automation tool, attach memory and debug controller. The debug debug controller which provides functionality for programming and debugging the processor, and a universal asynchronous receiver transmitter (UART).

1. From the IP Integrator, add the MicroBlaze processor to the block diagram
2. Using block automation, enable 128kB local memory, Debug and UART for Debug Module, and Peripheral AXI port.

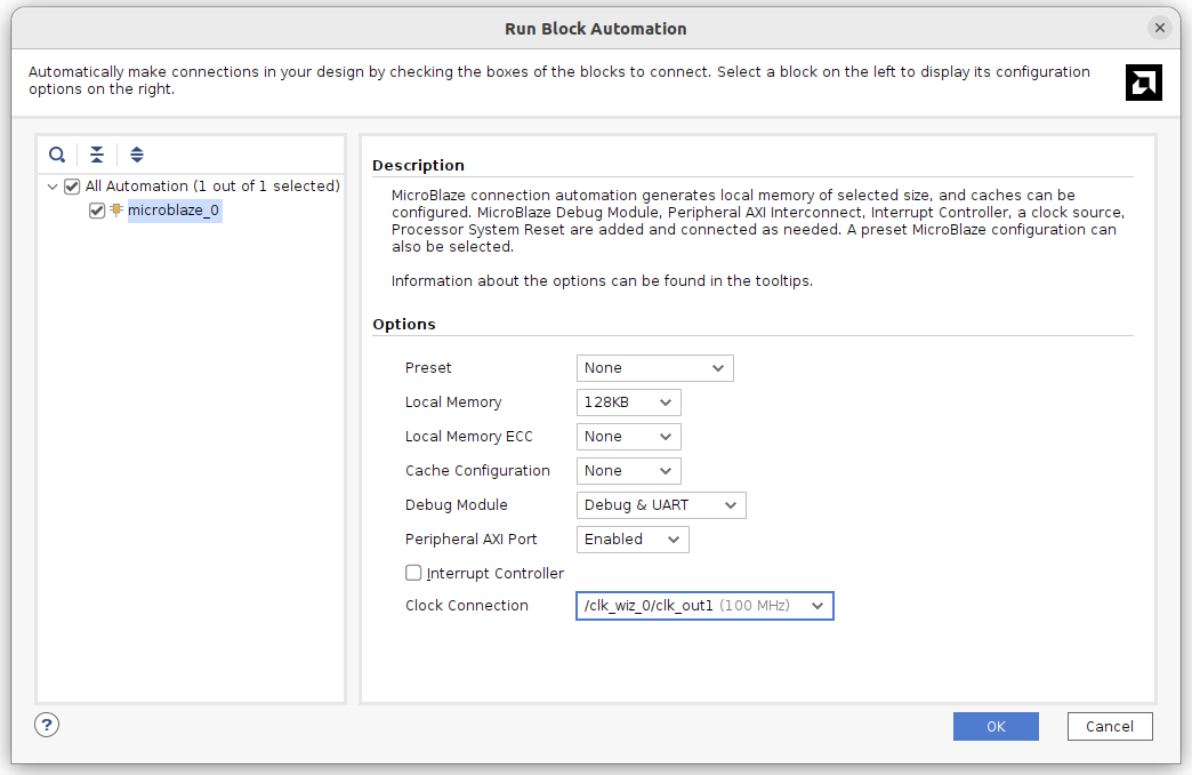

Figure 3: Recommended block automation configuration.

3. After wiring up all the miscellaneous connections, navigate to the address tab and check the memory map. A snippet of the memory map is illustrated in Figure 4 below.
    1. The base address of the local memory should correspond with the processor's reset vector, which by default is 0x0000_0000.
    2. The range of the instruction and data memories determine their actual memory. 128KB is sufficient for our purposes.
4. Go back to the block diagram and validate the design (checkbox symbol). If no errors occur generate the bitstream, and export the `.xsa` file (To export: File → Export → Export Hardware, and check the box "include bitstream").

The AMD documenation [2 p.6] provides further information about the block-based approach in Vivado.

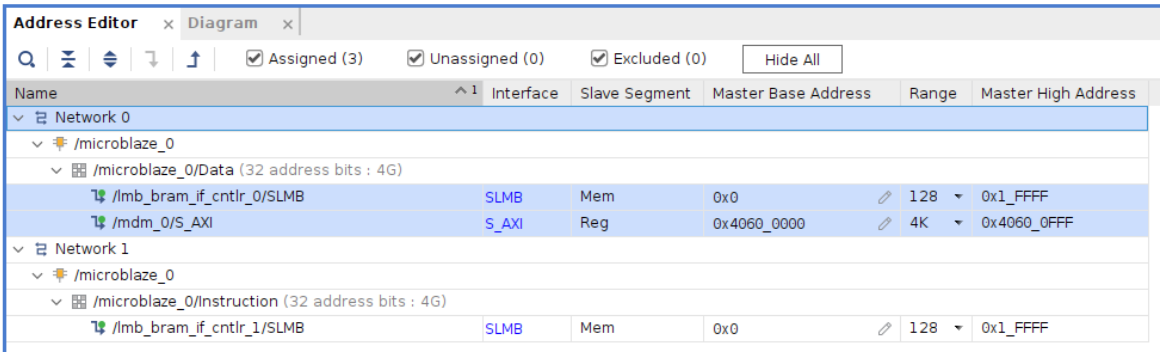

Figure 4: The expected memory map showing the shared instruction and data memory, as well as, the combined debug and uart peripheral.

Writing Software

We have created a hardware platform containing a microprocessor. The next step is to compile a hello world C application and run execute it on the hardware. We will use **Vitis classic** for this step.

The `.xsa` file generated in vivado contains the programming file for the FPGA, as well as metadata for the AMD tools to set up an embedded development environment in Vitis. The embedded development environemnts contains all the low low level libraries needed to boot the microprocessor, and manage basic hardware functions such as standard input and output.

Using Vitis create a hardware platform providing the `.xsa`. Afterwards create an application selecting the **hello_world** template. Build the project and start a debug session to run it on the target hardware. If everything is configured correctly, the program will execute successfully and the expected output will be displayed in the console.

The AMD documentation [3 p.18] provides further information about software development in Vitis.

AMD Tools on ZITI Workstations

The 2024.2 AMD tools are available on the zitipool workstations as an apptainer container. To use the tools, you must first start the container's shell with

```
apptainer shell /shares/ziti-opt/software/cat/vivado_2024_2.sif
```

Then you can launch vivado with **vivado**, and vitis with **vitis --classic**.

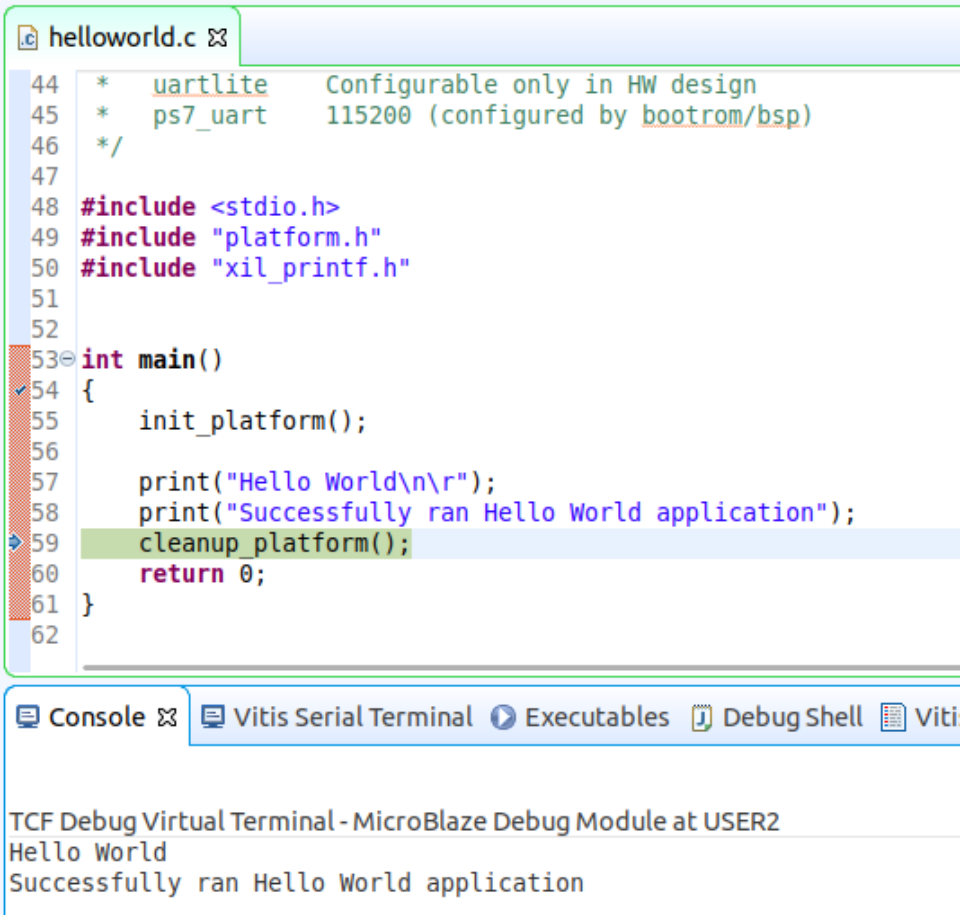

Figure 5: Illustration of the hello_world template in Vitis being debugged on hardware. The UART output is shown in the TCF Debug Virutal Terminal.

Deliverables

There are no deliverables or grading associated with this lab session. We will be using this project as basis for the next FPGA labs and homeworks.

References

1. MicroBlaze Processor Reference Guide (UG984)
2. Getting Started with Vivado IP Integrator (UG994)
3. Getting Started with Vitis (UG1400)
4. Zybo Z7 Reference Manual
5. Zybo Z7 Revision D.1 Schematic